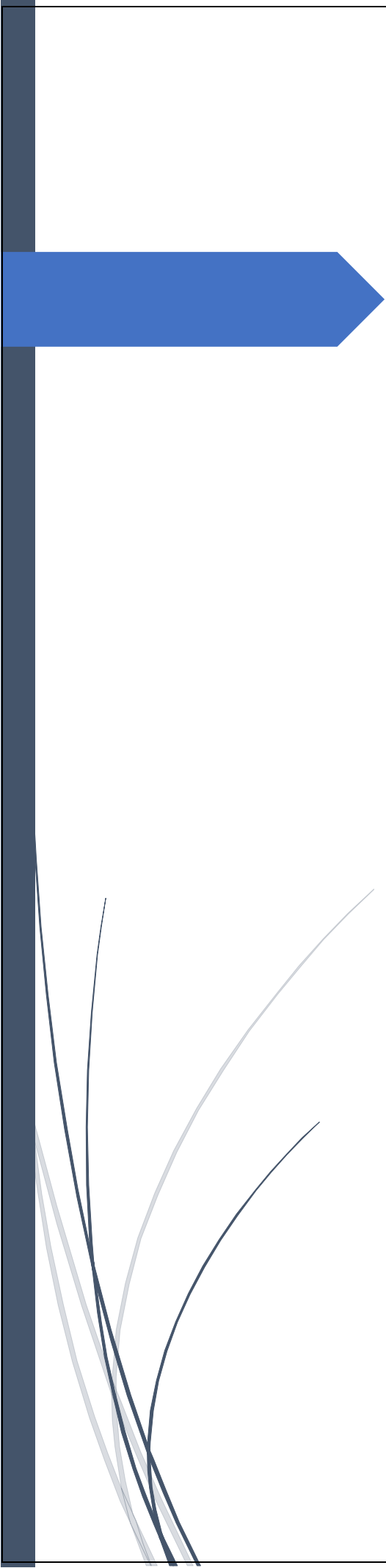




Assignment-1

Security in computing and IT

By, Patel Smit Mukeshbhai : S4137209

Anthony Mackenzy : S4129857

Table of Contents

❖ PASS - Q1 :	2
❖ PASS - Q2.....	4
❖ CREDIT – Q3:	10
❖ DI – Q4 :	13
❖ HD - Q5.....	16
❖ References :	19

❖ PASS - Q1 :

In order implement secured access, let's implement the one time security access key generation process and One-time pad generation process to check access. When the employee sends a request for the file access the system uses One Time Security access key generation process to generate access key. Which the system sends to the employee. Employee uses this access key to get access of the file.

➤ Algorithm for One time security access key generation process :

⇒ **Step 1 :** Generate 6 digit combined value.

Use the Employee ID, File ID and three 6 digit Secret codes of Oracle's members and XOR them to generate a 6 digit output.

$$\text{MiddleOutput} = \text{EmployeeID} \oplus \text{FileAccessID} \oplus S1 \oplus S2 \oplus S3$$

S1 = Information Security Manager's Secret Code

S2 = Information Security Director's Secret Code

S3 = Chief Information Security Officer's Secret Code

⇒ **Step 2 :** OTP Generation

Generate a random OTP (One-Time Pad).

⇒ **Step 3 :** Access Key Generation

Use the 6 digit output from the 1st step and XOR it with a random OTP to generate an access key.

$$\text{AccessKey} = \text{MiddleOutput} \oplus \text{OTP}$$

⇒ **Step 4 :** Store the OTP

Save the the OTP securely in the system for verification.

⇒ **Step 5 :** Send the Access Key.

Send the access key to the employee via SMS or Email.

➤ Example :

EmployeeID = 4137209

FileAccessID = 4129857

S1 = 424242

S2 = 123123

S3 = 666666

⇒ **Step 1 :**

$$\text{MiddleOutput} = \text{EmployeeID} \oplus \text{FileAccessID} \oplus S1 \oplus S2 \oplus S3$$

$$= 4137209 \oplus 4129857 \oplus 424242 \oplus 123123 \oplus 666666$$

$$= 009400 \oplus 424242 \oplus 123123 \oplus 666666$$

$$= 009400 \oplus 498113 \oplus 666666$$

$$= 507257 \oplus 666666$$

$$= 889171$$

⇒ **Step 2 :**

Random OTP = 125679

⇒ **Step 3 :**

AccessKey = MiddleOutput $\oplus$ OTP

$$= 889171 \oplus 125679$$

$$= 818108$$

⇒ **Step 4 :** Store the OTP (124679) instead of the Access key to prevent unauthorized access of the file.

⇒ **Step 5 :** Send the AccessKey (818108) to the employee.

➤ **Algorithm for One time Pad generation process to check access:**

Step 1 : Generate 6 digit middle value.

Use the 6 digit employee ID, File Access ID and three 6 digit Secret codes of Oracle's members and XOR them to generate a 6 digit output.

$$\text{MiddleOutput} = \text{EmployeeID} \oplus \text{FileAccessID} \oplus S1 \oplus S2 \oplus S3$$

S1 = Information Security Manager's Secret Code

S2 = Information Security Director's Secret Code

S3 = Chief Information Security Officer's Secret Code

Step 2 : Generate the Verification Code.

$$\text{VerificaitonCode} = \text{MiddleOutput} \oplus \text{AccessKey}$$

Step 3 : Compare the OTP stored in the system and Verification Code. If it's same then give access to the employee else deny the access.

If VerificaitonCode == OTP :

Allow access.

Else :

Deny access.

➤ **Example :**

EmployeeID = 4137209

FileAccessID = 4129857

S1 = 424242

S2 = 123123

S3 = 666666

AccessKey = 818108

⇒ **Step 1 :**

MiddleOutput = EmployeeID $\oplus$ FileAccessID $\oplus$ S1 $\oplus$ S2 $\oplus$ S3
= 4137209 $\oplus$ 4129857 $\oplus$ 424242 $\oplus$ 123123 $\oplus$ 666666
= 009400 $\oplus$ 424242 $\oplus$ 123123 $\oplus$ 666666
= 009400 $\oplus$ 498113 $\oplus$ 666666
= 507257 $\oplus$ 666666
= 889171

⇒ **Step 2 :**

VerificationCode = MiddleOutput $\oplus$ AccessKey
= 889171 $\oplus$ 818108
= 125679

⇒ **Step 3 :**

Compare OTP and the computed Verification code if both are same then give access to the file.

If VerificationCode == OTP :

Give Access.

Else :

Deny Access.

Here VerificationCode is same as the OTP, which means the employee will get access of the file. [1]

❖ PASS - Q2.

➤ Ciphertext

KBJQHQQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC KZW BKQXQZL
UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX DYXJ RKFKZGV QZZPSKJQPZ NQJM
BVXUPZXQRQFQJC, VZXYBQZL JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ
KFLPBQJMDX JMKJ XMKUV MYDKZ VOUBBQVZGVX KZW WVGQXQPZX. KX BDQJ XJYWVZJX
QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVVW NQJM GKYJQPZ

➤ Letter frequencies :

- Total letters without punctuations and spaces: 285
- Calculate frequency percentage: $\frac{\text{count}}{285} \times 100\%$

Rank	Cipher	Count	%
1	Q	37	12.9
2	Z	31	10.9
3	X	26	9.1
4	J	25	8.8
5	K	24	8.4
6	V	22	7.7
7	B	19	6.7
8	P	17	5.9
9	G	11	3.9
10	Y	10	3.5
11	F	9	3.2
12	W	8	2.8
13	C	7	2.5
14	U	7	2.5
15	M	7	2.5
16	L	6	2.1
17	D	5	1.8
18	H	4	1.4
19	R	4	1.4
20	S	2	0.7
21	N	2	0.7
22	I	1	0.4
23	O	1	0.4

- In the ciphertext, most frequent letters are Q and Z.

➤ Bigrams and short words :

- Bigram counts

Bigram	Count
KZ	8
ZX	8
QJ	8
QZ	7
PZ	7
JQ	5
XJ	4
FQ	4
VZ	5
QX	5

ZW	5
XQ	4
JM	5
VX	5
QP	5

- Smaller single words present in the ciphertext:
 - 2-letter words:
 - QX, KX, QJ, JP
 - 3-letter words:
 - KZW
- Recurring 3-letter word can be “THE”, “AND”, “FOR”, etc.
- Recurring 2-letter words can be “IT”, “IS”, “IN”, “OF, etc.

➤ Trial mapping table :

- Highest frequency cipher to highest frequency plaintext substitution:
 - Ciphertext:
 - KBJQHGGQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC
KZW BKQXQZL UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX
DYXJ RKFKZGV QZZPSKJQPZ NQJM BVXUPZXQRQFQJC, VZXYBQZL
JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ KFLPBQJMDX
JMKJ XMKUV MYDKZ VOUVBQVZGVX KZW WVGQXQPZX. KX BDQJ
XJYWVZJX QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVWW NQJM
GKYJQPZ
 - Plaintext:
 - ISOEOHEOIF ETONNIELINEN EA ISETAHPSDETI NSNCS ETWYAOSC ITE
SIETAD UNPHPTWE ONMOEIF YNINEOTZTA. PS LIETAEIOETZTA DYAIO
RIFINEEN ETTPSONIET NEMO SNAUNTAEEEROE, ENTAESIL
ISETAUIENSC, HIETENSN, ITE IGGPYTOIEROEI ET IFLPSOIMDA IMO I
AMIUN MYDIT NOUNSINEENIA ITE WINEOEEOETZTA. IA SDOE
AEYWINSIO EI EA CPSN SNAUNTAEEEROE IO UNPINEEN NEMO INYOET
 - No useful words found in the plaintext.
- Word shape driven hypothesis
 - 3-letter KZW and 2-letter KX/QX patterns.
 - **Test 1:** KZW = THE
 - K = T, Z = H, E = W
 - Then 2 letter bigram KZ = TH this is Possible because TH is common in English language.
 - **Test 2:** KZW = AND
 - K = A, Z = N, W = D

- AN and ND are common in English language, KZ and ZW which maps to AN and ND are common bigrams in the ciphertext.
- KX become A_: AT or AS possible.

➤ Applying Test 1 :

- Ciphertext:
 - KBJQHGGQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC KZW BKQXQZL UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX DYXJ RKFKZGV QZZPSKJQPZ NQJM BVXUPZXQRQFQJC, VZXYBQZL JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ KFLPBQJMDX JMKJ XMKUV MYDKZ VOUVBQVZGVX KZW WVGQXQPZX. KX BDQJ XJYWVZJX QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVVW NQJM GKYJQPZ
- Plaintext:
 - TBJQHGGQTF QHJVFFQLVHGV QX JBTXHHPBDQHL VSVBC QHEYXJBC THE BTQXQHL UBPHPYHE VJMQGTF IYVXJQPHX. PBLTHQXTJQPHX DYXJ RTFT HGV QHHPS TJQPH NQJM BVXUPHXQRQFQJC, VHXYBQHL JBTXHUTBVHGC, HTQBH VXX, THE TGGPYHJ TQRQFQJC QH TFLPBQJMDX JMTJ XMTUV MYDTH VOUVBQHVGXVX THE EVGQXQPHX. TX BDQJ XJYHV TJX QJ QX CPYB BVXUPHXQRQFQJC JP UBPGVV E NQJM TGYJQPH
- No visible interesting words found in the plain text.

➤ Applying Test 2 :

- Ciphertext:
 - KBJQHGGQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC KZW BKQXQZL UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX DYXJ RKFKZGV QZZPSKJQPZ NQJM BVXUPZXQRQFQJC, VZXYBQZL JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ KFLPBQJMDX JMKJ XMKUV MYDKZ VOUVBQVZGVX KZW WVGQXQPZX. KX BDQJ XJYWVZJX QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVVW NQJM GKYJQPZ
- Plaintext:
 - ABJQHGGQAF QNJVFFQLVNGV QX JBANXHPBDQN LSVBC QNDYXJBC AND BAQXQNL UBPHPYND VJMQGAF IYVXJQPNX. PBLANQXAJQPNX DYXJ RAFANGV QNNPSAJQPN NQJM BVXUPNXQRQFQJC, VNXYBQN JBANXUABVNGC, HANBAVXX, AND AGGPYANJARQFQJC QN AFLPBQJMDX JAMJ XMAUV MYDAN VOUVBQN NGVX AND DVGQXQPNX. AX BDQJ XJYNV AJX QJ QX CPYB BVXUPNXQRQFQJC JP UBPGVVD NQJM GAYJQPN
- Assuming frequent 2-letter combination "QJ QX" as "IT IS" because "IT IS" is a common consecutive words in English language:
 - K = A, Z = N, W = D, Q = I, J = T, X = S

- Ciphertext:
 - KBJQHGGQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC KZW BKQXQZL UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX DYXJ RKFKZGV QZZPSKJQPZ NQJM BVXUPZXQRQFQJC, VZXYBQZL JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ KFLPBQJMDX JMKJ XMKUV MYDKZ VOUVBQVZGVX KZW WVGQXQPZX. KX BDQJ XJYWVZJX QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVVW NQJM GKYJQPZ
- Plaintext:
 - ABTIHIGIAF INTVFFILVNGV IS TBAINSHPBDIN VSVBC INDSYTBC AND BAISINL UBPHPYND VTMIIGAF IYVSTIPNS. PBLANSAITIPNS DYST RAFA NGV INNP SAITIPN NITM BVSPUNSIRIFITC, VNSYB INL TBAINSUA BVNGC, HANTAVSS, AND AGGPYANTARIFITC IN AFLPBITMDS TAMT SMAUV MYDAN VOUVBIN NGVS AND DVGISIPNS. AS BDIT STYNV TIS IT IS CPYB BVSPUNSIRIFITC TP UBPGVVD NITM GAYTIPN
- Most frequent cipher letters are already mapped. Next most frequent cipher-letter is V. Mapping V to the most frequent English letter E.
- Ciphertext:
 - KBJQHGGQKF QZJVFFQLVZGV QX JBKZXHPBDQZL VSVBC QZWYXJBC KZW BKQXQZL UBPHPYZW VJMQGKF IYVXJQPZX. PBLKZQXKJQPZX DYXJ RKFKZGV QZZPSKJQPZ NQJM BVXUPZXQRQFQJC, VZXYBQZL JBKZXUKBVZGC, HKQBZVXX, KZW KGGPYZJKRQFQJC QZ KFLPBQJMDX JMKJ XMKUV MYDKZ VOUVBQVZGVX KZW WVGQXQPZX. KX BDQJ XJYWVZJX QJ QX CPYB BVXUPZXQRQFQJC JP UBPGVVW NQJM GKYJQPZ
- Plaintext:
 - ABTIHIGIAF INTEFFILENGE IS TBANSHPBDINL ESEBC INDYSTBC AND BAISINL UBPHPYND ETMIGAF IYESTIPNS. PBLANISATIPNS DYST RAFANGE INNPSATIPN NITM BESUPNSIRIFITC, ENSYBINL TBANSUA BENG C, HAIBNESS, AND AGGPYNTARIFITC IN AFLPBITMDS TMAT SMAUE MYDAN EOUEBIENGES AND DEGISIPNS. AS BDIT STYDENTS IT IS CPYB BESUPNSIRIFITC TP UBPG EED NITM GAYTIPN
- Now there are some words we can guess:
 - "ESEBC" = "EVERY"
 - S = V, B = R, C = Y
- Plaintext:
 - ARTIHIGIAF INTEFFILENGE IS TRANSHPRDINL EVERY INDYSTRY AND RAISINL URPHPYND ETMIGAF IYESTIPNS. PRLANISATIPNS DYST RAFANGE INNPVATIPN NITM RESUPNSIRIFITY, ENSYRINL TRANSUA RENGY, HAIRNESS, AND AGGPYNTARIFITY IN AFLPRITMDS TMAT SMAUE MYDAN EOUE RIENGES AND DEGISIPNS. AS RDIT STYDENTS IT IS YPYR RESUPNSIRIFITY TP URPGEED NITM GAYTIPN

➤ Mapping obvious words :

- Current plaintext:
 - ARTIHIGIAF INTEFFILENGE IS TRANSHPRDINL EVERY INDYSTRY AND RAISINL URHPYND ETMIGAF IYESTIPNS. PRLANISATIPNS DYST RAFANGE INNPVATIPN NITM RESUPNSIRIFITY, ENSYRINL TRANSUARENGY, HAIRNESS, AND AGGPYNTARIFITY IN AFLPRITMDS TMAT SMAUE MYDAN EOUERIENGES AND DEGISIPNS. AS RDIT STYDENTS IT IS YPYR RESUPNSIRIFITY TP URPGEED NITM GAYTIPN
- Mapping words:
 - INTEFFILENGE = INTELLIGENCE
 - INDYSTRY = INDUSTRY
 - RAISINL = RAISING
 - INNPVATIPN = INNOVATION
 - STYDENTS = STUDENTS
- Plaintext:
 - ARTIHICIAL INTELLIGENCE IS TRANSHORDING EVERY INDUSTRY AND RAISING UROHOUND ETMICAL IUESTIONS. ORGANISATIONS DUST BALANCE INNOVATION NITM RESUONSIRILITY, ENSURING TRANSUARENCY, HAIRNESS, AND ACCOUNTARILITY IN ALGORITMDS TMAT SMAUE MUDAN EOUERIENCES AND DECISIONS. AS RDIT STUDENTS IT IS YOUR RESUONSIRILITY TO UROCEED NITM CAUTION
- Repeating the above step, we can now map more words:
 - ARTIHICIAL = ARTIFICIAL
 - TRANSHORDING = TRANSFORMING
 - ETMICAL = ETHICAL
 - IUESTIONS = QUESTIONS
- Now we can fill the rest according to the context.
- Final substitution:

Cipher	Plain
B	R
C	Y
D	M
F	L
G	C
H	F
I	Q
J	T
K	A

L	G
M	H
N	W
O	X
P	O
Q	I
R	B
S	V
U	P
V	E
W	D
X	S
Y	U
Z	N

- Final decrypted plaintext:

ARTIFICIAL INTELLIGENCE IS TRANSFORMING EVERY INDUSTRY AND RAISING PROFOUND ETHICAL QUESTIONS. ORGANISATIONS MUST BALANCE INNOVATION WITH RESPONSIBILITY, ENSURING TRANSPARENCY, FAIRNESS, AND ACCOUNTABILITY IN ALGORITHMS THAT SHAPE HUMAN EXPERIENCES AND DECISIONS. AS RMIT STUDENTS IT IS YOUR RESPONSIBILITY TO PROCEED WITH CAUTION. [2]

❖ CREDIT – Q3:

Given values :

$n = 10009999019$ (multiplication of two prime numbers p and q)

$e = 65537$ (public key)

(a) Let's find primes p and q such that $n = p \cdot q$.

Using online prime factor we got,

$$p = 99991 \text{ and } q = 100109$$

let's check if p and q is correct or not,

$$p \cdot q = 99991 \cdot 100109 = 10009999019 = n$$

that means p and q are correct.

Compute the private key (d)

Euler's totient,

$$\begin{aligned}\phi(n) &= (p-1)*(q-1) \\ &= (99991-1)*(100109-1) \\ &= (99990)*(100108) \\ &= 10009798920\end{aligned}$$

Public key (e) should be a prime number and it must be less than the totient ($\phi(n)$).

Also e and $\phi(n)$ both must be coprime (no common factor other than 1).

Public key (e) follows all this rules.

Now let's compute the private key (d). The Product of the Public Key and the Private Key when divided by the Totient, must result in a remainder of 1. Or, to put it simply, the following formula must be true:

$$(d*e) \text{ MOD } \phi(n) = 1$$

From this formula we can compute the formula $d = e^{-1}(\text{MOD } \phi(n))$

Here we got,

$$e = 65537$$

$$\phi(n) = 10009798920$$

$$d = 65537^{-1}(\text{MOD } 10009798920)$$

$$= 6509721833$$

Let's check if d is right or not using the following formula

$$(d*e) \text{ MOD } \phi(n) = 1$$

$$= (6509721833*65537) \text{ MOD } 10009798920$$

$$= 426627639769321 \text{ MOD } 10009798920$$

$$= 1$$

We got 1 as a result which shows that the value of d is right. [3] [4]

(b) ciphertext $C = 1880434495$

let's compute the original plaintext message (M) using the decryption formula,

$$M = C^d \bmod n$$

Here $C = 1880434495$,

$$d = 6509721833$$

$$n = 10009999019$$

let's compute the plaintext M ,

$$\begin{aligned} M &= 1880434495^{6509721833} \bmod 10009999019 \\ &= 12345 \end{aligned} \quad [3] \quad [4]$$

(c) Let's verify if the secret message is correct or not. For this encrypt the plaintext M using the public key (e) and compare if it's equal to the ciphertext (C).

$$C' = M^e \bmod n$$

Here $M = 12345$

$$e = 65537$$

$$n = 10009999019,$$

$$\begin{aligned} C' &= 12345^{65537} \bmod 10009999019 \\ &= 1880434495 \end{aligned}$$

Here encrypted text (C') = cipher text (C), decryption verified successfully.

[3] [4]

(d) Yes, I would have find the private key faster if I had 10 computers. In the process of finding the private key the prime factorization of n is the hardest. If we just divide the work between 10 computers, where each one checks different portion of numbers than we can make the process much faster – almost 10 times faster than using a single computer. We can run algorithms such as Pollard's Rho, Trail Division, etc simultaneously to find the prime factors very quickly. [5] [6]

❖ DI – Q4 :

- (a) The main objective of the blind bidding system is to stop the bidders from seeing each other's bids during the bidding phase. In the bidding phase bidders only share hash values instead of the actual bid and after the bidding phase each bidder shares their actual bid values with the system. The system then uses this bid values to compute hash value for each bid to verify them. Lastly, it compares all the bids and finds the bid ,which is the smallest to declare the winner.

➤ Process :

1. Bidder bids some amount. (e.g., \$500)
2. Client side (bidder's system) computes the hash value using the SHA-256.

Example :

Bid = \$ 500

SHA-256("500")

= 0604cd3138feed202ef293e062da2f4720f77a05d25ee036a7a01c9cfcd1f0a

3. Client (bidder) shares the hash value with the system (Server).
4. Server stores the hash value and the bidder's identity.
5. After the bidding process closes, bidders share their plaintext bid. The system then verifies each bid by computing the hash value for all of them and matches the computed hash values with the stored hash values respectively.

➤ Example with the numeric values :

Bidder	Plaintext Bid	SHA-256 Hash
Bidder_1	500	0604cd3138feed202ef293e062da2f4720f77a05d25ee036a7a01c9cfcd1f0a
Bidder_2	800	1a1cf797fabe7f95836fabeca626907c77b3e6c9aff7c2290b396a238c69362e
Bidder_3	688	1c63ed9164d61acfd1f4f3a7b6dfacbd98d1dc01e755b7b558c6af0491154a2e
Bidder_4	800	1a1cf797fabe7f95836fabeca626907c77b3e6c9aff7c2290b396a238c69362e

6. The system then decides the winner by comparing the bids and finding the lowest one.

(b) Yes, a smart bidder can exploit the system using the dictionary attack. We can see that different bidders with same bidding value also gets the same hash values makes it vulnerable as the attacker can see that two bidders have entered the same values. Usually Hashing functions are defined by three fundamental properties :

1. One-way property : This means that it is computationally infeasible to reverse the process and find the original text data from it's hash value. This is a crucial feature that protects the sensitive information, such as passwords, by ensuring that even if the hash value is compromised, the original data remains secure.
2. Second Preimage resistance : This property states that for the given specific input, it is computationally infeasible to find a second, distinct input value that can generate the same hash.
3. Collision Resistance : This property ensures that it is computationally infeasible to find any two different values that produces the same hash value. While collisions are theoretically possible due to the fixed-size output of the hash function, a strong cryptographic algorithm minimizes this probability.

There is another important characteristic, which is the “avalanche effect”, where a minor change in the input (altering a single character) can lead to drastic changes in the output. This is why hash functions can be used to verify the integrity of the data.

This properties actually makes the hash functions secure. But they are still vulnerable to dictionary attacks. Lets see this in the blind bidding system. This system is vulnerable due to the fact that the input range (bids) are usually small integers like (1 to 5000). This is why, it's really easy to predict the input values. [7]

➤ **This is how a malicious bidder can exploit this system :**

1. pre-compute hashes for all the possible input values (range from 1 to 5000) and create a hash-table.
2. Collect all the bids submitted by other bidders.
3. Compare the hash values collected from the bidders with the hash table to find the exact bid amount.

➤ **Example :** Hash-table generated by the attacker.

Plaintext Bid	SHA-256 Hash
1	6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b
....	
....	
1500	9f69998560dcfd8016442e0a32e959191df095817a164ce844c64ec5a8b0cc1b
1501	3873ecf4855ce373db4b7afc32fe5884d984d37a32c2c605065671d3eed825d4
....	
....	
5000	0f8eb4b72b6e0c9e88b388eb967b49e067ef1004bf07bffc22c3acb13b43580a

Then attacker will collect the hashes of the other bidders to compare them with this hash-table and find the exact value.

➤ **Example :**

Other bidder's hash :

3873ecf4855ce373db4b7afc32fe5884d984d37a32c2c605065671d3eed825d4

By comparing this hash value with the table we can see that plaintext bid for this hash is 1501. Now the malicious bidder gets all the bids and then submits lower bid than others to win. [8]

(c) To prevent the dictionary attack or other pre-computation attacks, the system must use the salting technique. In this technique, system generates a unique random salt and adds it to the plaintext bid. Then it computes the hash of the concatenated string. This removes the problem of getting the same hash values for the same input as we add the unique salt to the input to compute the hash, which creates a unique string for every input.

➤ **Improved Process :**

1. When a bidder submits a bid, the system generates a unique random salt (random string).
2. Concatenate the bid and the salt to create a new string.

Hash $H = \text{SHA-256}(\text{bid} || \text{salt})$

For example,

bid = 900,

salt = fgjk12

input = 900fgjk12

$H = \text{SHA-256}("900fgjk12")$

= e222c1b675a10a8cfbf6ff4fa4f7d945fd61a274b907578d9f72e1768481d277

3. Bidder sends the hash value to the system.
4. System stores both the hash and the salt.
5. After the bidding phase, bidder shares the salt + bid, then system computes the hash value and compares it with the stored hash value to verify the bid. This happens for all the bidders.
6. The system then decides the winner by comparing the bids and finding the lowest one

➤ Example with the numeric values :

Bidder	Plaintext Bid	Unique Salt	Concatenated String	SHA-256 Hash
Bidder_1	500	aBcde1	500aBcde1	cd1e505b7a0bddc809311980b465f84aafd4807d145bf07f29c4f1ba9eff7a49
Bidder_2	800	Vxk123	800Vxk123	baf1d54baddfeb6cb4c8373d152670dcc21f78211591eba35e2492139da9384
Bidder_3	688	Lsdfgh	688Lsdfgh	78ecfbce08caacddae82eca22160911d67f42ef8f56c62c29cd3abc3ee1b8ca0
Bidder_4	800	CCie12	800CCie12	877f904b1b726e623b4fd3d82a4d4014f5d09e63dd63b1d2b0da6916c76c09a6

We can see that we get different outputs for the same input values (e.g., 800). This way the attacker will have to generate different hash tables for each bidder as we use unique salt with each input. Which makes it more secured than the hashing function without the salt. [9]

❖ HD - Q5

Student ID: S4129857

Account ID: M=412985

a)

Choose two prime numbers,

$$p = 3217$$

$$q = 3041$$

$$\text{Public exponent (e)} = 773$$

compute modulus n,

$$n = p \times q$$

$$n = 3217 \times 3041$$

$$n = 9782897$$

Compute Euler's totient,

$$\phi(n) = (p - 1)(q - 1)$$

$$\phi(n) = (3217 - 1)(3041 - 1)$$

$$\phi(n) = 3216 \times 3040$$

$$\phi(n) = 9776640$$

Select public exponent (e),

$e = 773$ such that $\gcd(e, \phi(n)) = 1$

compute Private key: d ,

$$d \cdot e \equiv 1 \pmod{\phi(n)}$$

$$d \cdot 773 \equiv 1 \pmod{9776640}$$

$$d = 5147597 \text{ (using modular inverse of } e \text{ mod } \phi(n))$$

Public Key: $(n, e) = (9782897, 773)$

Private Key: $(n, d) = (9782897, 5147597)$

Let's encrypt the message,

Plaintext $M = 412985$

Ciphertext ,

$$C = M^e \pmod{n}$$

$$C = 412985^{773} \pmod{9782897}$$

$$C = 4517206$$

Now Let's Decrypt the ciphertext,

$$M' = C^d \pmod{n}$$

$$M' = 4517206^{5147597} \pmod{9782897}$$

$$M' = 412985 \text{ [3][4]}$$

b)

Choose public parameters,

Prime modulus: $p = 8902967$

Generator: $g = 3707$

Select Private Key,

Private key: $x = 7841$

Compute public key component,

$$y = g^x \pmod{p}$$

$$y = 3707^{7841} \pmod{8902967}$$

$$y = 795883$$

Public Key: $(p, g, y) = (8902967, 3707, 795883)$

Choose Random Integer r ,

$$r = 16889$$

Encrypt Message,

Plaintext $M = 412985$

$$C_1 = g^r \bmod p$$

$$C_1 = 3707^{16889} \bmod 8902967$$

$$C_1 = 3079929$$

$$C_2 = M \cdot y^r \bmod p$$

$$C_2 = 412985 \cdot 795883^{16889} \bmod 8902967$$

$$C_2 = 5826446$$

Ciphertext sent to user,

$$C = (C_1, C_2) = (3079929, 5826446)$$

Decrypt ciphertext :

$$M' = C_2 \cdot (C_1^x)^{-1} \bmod p$$

$$C_1^x = 3079929^{7841} \bmod 8902967$$

$$C_1^x = 2508500$$

$$(C_1^x)^{-1} \bmod p = 2508500^{-1} \bmod 8902967$$

$$(C_1^x)^{-1} = 1654589$$

$$M' = 5826446 \cdot 1654589 \bmod 8902967$$

$$M' = 412985$$

Decryption successful $M' = M$ [10]

❖ References :

- [1] A. Froehlich, "What is one-time pad? - Definition from WhatIs.com," *SearchSecurity*, Jan. 2022.
<https://www.techtarget.com/searchsecurity/definition/one-time-pad>
- [2] D. Rodriguez-Clark, "Frequency Analysis: Breaking the Code," *Crypto Corner*, 2017. <https://crypto.interactive-maths.com/frequency-analysis-breaking-the-code.html>
- [3] GeeksforGeeks, "RSA Algorithm in Cryptography," *GeeksforGeeks*, Apr. 22, 2017. <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>
- [4] "RSA Algorithm Example," *www.cs.utexas.edu*.
<https://www.cs.utexas.edu/~mitra/honors/soln.html>
- [5] "Integer Factorization Algorithms: A Comparative Analysis," *softwaredominos.com*. <https://softwaredominos.com/home/science-technology-and-other-fascinating-topics/integer-factorization-algorithms-a-comparative-analysis/>
- [6] R. Brent, "Parallel Algorithms for Integer Factorisation." Available:
<https://maths-people.anu.edu.au/brent/pd/rpb115.pdf>
- [7] GeeksforGeeks, "Cryptography Hash Functions," *GeeksforGeeks*, May 14, 2024. <https://www.geeksforgeeks.org/competitive-programming/cryptography-hash-functions/>
- [8] B. Wolford, "What is a rainbow table attack and how to prevent it?," *Proton*, May 31, 2024. <https://proton.me/blog/what-is-rainbow-table-attack>

[9] Tshedimoso Makhene, "What is password salting?," *Paubox.com*, Feb. 10, 2024. <https://www.paubox.com/blog/what-is-password-salting>

[10] GeeksforGeeks, "ElGamal Encryption Algorithm," *GeeksforGeeks*, Nov. 11, 2018. <https://www.geeksforgeeks.org/computer-networks/elgamal-encryption-algorithm/>